

Unit-4 UNIX OPERATING SYSTEM

Syllabus:

Introduction to Unix System: The Unix Operating System, The UNIX architecture.

Shell Programming: Vi editor, shell types, shell command line processing, shell script features, executing a shell script, system and user-defined variables, expr command, shell screen interface, read and echo statement, command substitution, escape sequence characters, shell script arguments, positional parameters, test command, file test, string test, numeric test.

Conditional Control Structures: if statement, case statement Looping Control Structure-while, until, for, statements. Jumping Control Structures – break, continue, exit. Shell Programs covering the above concepts.

Unit-4 Chapter-1

Introduction to Unix System

Introduction to UNIX System

UNIX is a powerful and widely used Operating System. It was initially developed in 1970 by Ken Thompson and Dennis Ritchie at AT&T Bell Laboratories.

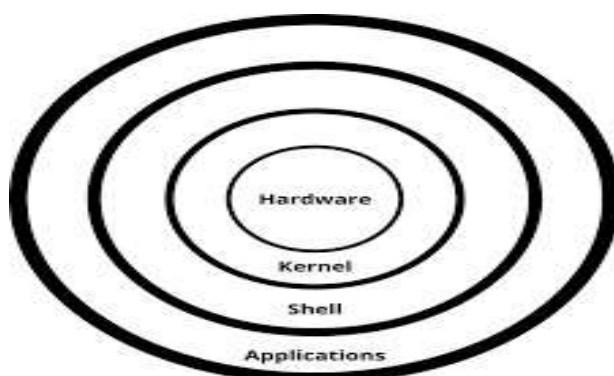
Definition:

UNIX is an operating system that acts as a interface between the user and the computer hardware.

UNIX is an operating system that allows multiple users to work at the same time and run many programs together while managing computer resources like CPU, memory, and files.

UNIX Architecture

The UNIX system is organized into the following layers:



1. Hardware

Hardware is the lowest and least powerful layer in the UNIX architecture. It consists of all the physical components of the computer that are visible and tangible.

- **CPU (Central Processing Unit):**

The CPU is the brain of the computer that processes data and controls all activities.

- **Motherboard:**

The motherboard is the main circuit board that connects all hardware components.

- **Memory:**

Memory is used to store data and instructions in a computer.

Examples: RAM, ROM

- **Input Devices:**

Input devices are used to enter data into the computer.

Example: Keyboard, Mouse

- **Output Devices:**

Output devices are used to display results from the computer.

Example: Monitor, Printer

- **Storage Devices:**

Storage devices are used to store data permanently.

Example: Hard Disk, Pen Drive

- **Peripheral Devices:**

Peripheral devices are external devices connected to the computer for input, output, or storage.

Example: Scanner, Printer

2. Kernel

The core part of the operating system, managing hardware resources.

The kernel is a program in the Operating System software that keeps on running in the background when the machine is on and it is responsible for allocating the resources to the processes, handling and coordinating the multiple processes running together.

In short, the kernel controls and coordinates all activities between software and hardware.

3. Shell

It Acts as a command interpreter, taking user commands and executing them by communicating with the kernel.

However, since the kernel is such an essential program in the Operating System and performs almost all of the major tasks, it should not be directly accessible to the user. So, there is a program between the user and the kernel that helps the user interact with the kernel. This program is called the shell.

Shell is an interpreter program that interprets the commands entered by the user and then sends the requests to the kernel to execute those commands. When the execution of the process is completed, the shell again sends a request to the kernel to display the program or information on the screen to the user.

So,

- Kernel is the interface between hardware and shell
- Shell is the interface between user and kernel

The shell is used for opening files, editing files, executing programs, and running system commands.

4. Applications

Applications are user-level or system-level programs such as editors, compilers, and utilities.

Applications access hardware resources through system calls, which are special functions provided by the kernel. These system calls are mostly written in the C programming language.

Unit 4 – Chapter 2

Shell Programming

SHELL PROGRAMMING

The shell provides an interface between the user and the UNIX operating system. It accepts input from the user, interprets commands, executes programs, and displays output. The shell is an environment in which users can run commands, programs, and shell scripts.

VI EDITOR

The vi editor (Visual Editor) is a powerful text editor used to create and modify files in UNIX and UNIX-like operating systems.

Modes of vi Editor

1. Command Mode (Default Mode)

- This is the default mode of vi when you open a file. This is the default mode of vi when you open a file. You cannot type text here.
- You can move the cursor, delete text, search, copy, paste, undo/redo, and perform other tasks without directly editing the text.

How to Enter: vi starts in Command Mode. If you're in Insert or Last Line Mode, pressing Esc will bring you back to Command Mode.

Cursor Movement

h - Move left
l - Move right
j - Move down
k - Move up
w - Move to next word
b - Move to previous word

Editing Commands

dd - Delete current line
dw - Delete a word
x - Delete one character
yy - Copy current line
p - Paste after cursor
P - Paste before cursor
U - Undo last action
.- Repeat last command

2. Insert Mode

- This mode allows you to **insert or modify text** in the file. You can type freely in this mode.
- Any character you type will be inserted directly into the file at the cursor's position.

How to Enter:

- i - Insert before cursor
- a - Insert after cursor
- I - Insert at beginning of line
- A - Insert at end of line
- o - New line below
- O - New line above

How to Exit: Press Esc to return to Command Mode.

3. Last line mode

Last line mode is used to **perform file-level operations** such as saving the file, quitting the editor, searching for text, and setting options.

This mode is entered by pressing the colon (:) key while in command mode.

How to Enter:

:w - Save file

:q - Quit

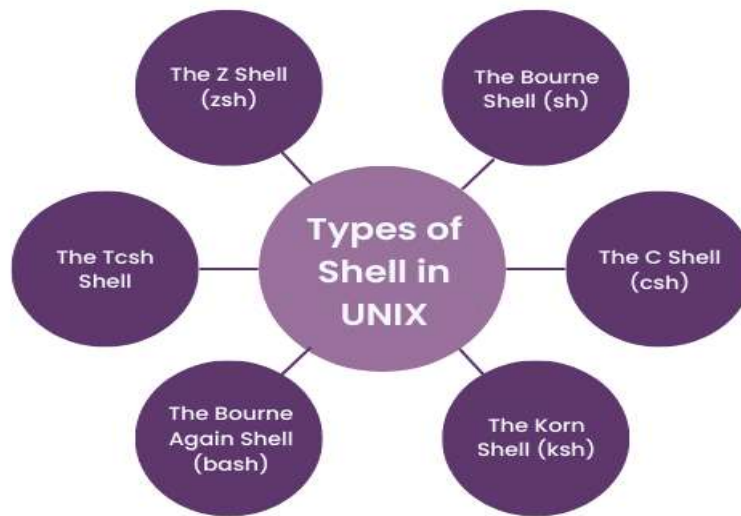
:wq - Save and quit

:/word - Search Commands

Advantages of vi

- Available in all UNIX/Linux systems
- Works without mouse
- Very fast for editing
- Used by system administrators and programmers

TYPES OF SHELLS



Bourne Shell (sh):

- It was the first shell Introduced in 1979 developed for the Unix operating system and is considered the foundation for many subsequent shells.
- The Bourne Shell was designed to be simple and efficient, with the ability to work across different
- Unix-based systems, making it highly portable.
- The Bourne Shell provides basic functionality such as **variables, control structures (like loops and conditionals), and built-in operators.**

C Shell (csh):

- Developed in the late 1970s, which resembles the C programming language, and was designed to improve the user experience compared to the Bourne Shell.
- It introduced interactive features like **job control, command history, and aliases**, which made it more user-friendly for interactive use compared to the Bourne Shell.
- However, it is less commonly used for scripting tasks compared to other shells.

Korn Shell (ksh):

- Created in the early 1980s, the Korn Shell combines the best features of the Bourne Shell and the C Shell, making it a versatile option.
- It introduced advanced features like **associative arrays, improved string manipulation, and enhanced scripting capabilities.**
- The Korn Shell became the default shell for many commercial UNIX systems due to its adaptability and powerful features.

Bourne Again Shell (bash):

- Released in 1989 as part of the GNU project, bash is an enhanced version of the Bourne Shell.
- It is widely known for its user-friendly features, such as **command history, auto-completion, and advanced scripting capabilities.**
- It is the default shell on most Linux systems and macOS, offering an extensive scripting environment **with better error handling, variable management, and scripting capabilities.**

Tcsh Shell:

- Tcsh is an improved version of the C Shell, developed in the late 1980s, It addresses many of the limitations of the original C Shell and introduces additional features, making it more powerful for interactive use and scripting.
- It addressed many of the **limitations of the original C Shell**, enhancing the interactive user experience.
- Tcsh is less widely used for scripting compared to shells like Bash or Korn Shell.

Z Shell (zsh):

- Released in 1990, the Z Shell It combines features from other shells like bash, ksh, and tcsh.
- It includes features like shared **command history, spelling correction** providing greater flexibility.
- Zsh is known for its user-friendly features like **auto-correction for misspelled commands and advanced tab completion**, which enhance productivity.

SHELL COMMAND LINE PROCESSING:

Shell Command Line Processing refers to the **series of steps** the shell follows when interpreting and executing commands entered by the user.

1. **Tokenization:** Splits the command into tokens (command, options, arguments).
2. **Interpretation:** Shell analyses and understands the each tokens and prepares them for execution and variables are expanded.
3. **Execution:** Executes the command and displays the output.

1. Tokenization:

When a user types a command into the shell, the shell first splits the command line into smaller units called tokens.

These tokens represent the command, options, arguments, and special characters.

OPERATING SYSTEM

Ex: `ls -l /home/user`

`ls` = command

`-l` = option

`/home/user` = Arguments

2. Interpretation:

After tokenization, the shell performs interpretation. In this step, the shell analyses and understands each token and prepares them for execution.

During interpretation, the shell checks the syntax of the command, expands variables, processes wildcards, and performs command substitution if present. Ex: `*` or `?`, `echo $(date)`, `$( )`

This step ensures that the command is correctly understood and ready to be executed by the system.

3. Execution:

Once the shell has processed the tokens and interpreted them, it proceeds to execute the command. If the command is internal, like `echo` or `cd`, the shell directly handles it.

For external commands, like `ls` or `cat`, the shell locates the executable file in directories listed in the `PATH` environment variable and runs it.

The result or output of the command is then displayed to the user or processed further, depending on the command.

SHELL SCRIPT FEATURES

1. Command Automation

Command automation means that a shell script can execute many commands automatically by running a single script file. This helps to save time and reduces manual work.

Example:

```
#!/bin/bash
```

```
date
```

```
cal
```

```
whoami
```

2. Easy to Learn

Shell scripting is easy to learn because it uses simple commands that are already used in the terminal. Even beginners can write and understand shell scripts.

Example:


```
echo "Welcome to Shell Scripting"
```

3. Variable Support

Variable support means that shell scripts can store values such as numbers or names in variables and use them later in the program.

Example:

```
name="Anu"  
echo "My name is $name"
```

4. Decision Making

Decision making means that a shell script can perform different actions based on a condition using if statements.

Example:

```
num=10  
if [ $num -gt 5 ]  
then  
    echo "Number is greater than 5"  
fi
```

5. Looping

Looping means that a shell script can repeat the same set of commands multiple times using loops.

Example:

```
for i in 1 2 3  
do  
    echo $i  
done
```

6. Command Line Arguments

Command line arguments mean that a shell script can accept values when the script is executed.

Example

```
echo "First value is $1"  
echo "Second value is $2"
```

7. User Input and Output

User input and output mean that a shell script can take input from the user and display output on the screen.

Example:

```
echo "Enter your name:"  
read name  
echo "Hello $name"
```

8. File and Directory Handling

File and directory handling means that a shell script can create, delete, and manage files and directories.

Example:

```
mkdir demo  
touch demo/file1.txt
```

9. Error Handling

Error handling means that a shell script can check whether a command is successful or not using the exit status.

Example:

```
ls file.txt  
if [ $? -eq 0 ]  
then  
    echo "File exists"  
else  
    echo "File does not exist"  
fi
```

10. Portability

Portability means that a shell script written on one UNIX or Linux system can run on other systems without modification.

Example:

A Bash script written on Linux can also run on UNIX systems

EXECUTING A SHELL SCRIPT

Executing a Shell Script means running a set of commands written in a script file to perform automated tasks in a Unix-like operating system.

To execute a shell script, follow these steps:

1. Make the Script Executable:

Before executing a shell script, you need to make it executable by changing its permissions.

```
Chmod +x script_name.sh
```

2. Run the Script:

Once the script is executable, you can run it in the terminal.

```
./script_name.sh    # For scripts in the current directory
```

or

```
bash script_name.sh  # Using the bash interpreter explicitly
```

3. Execute the Script with Full Path:

If the script is in a different directory, you can use the full path to execute it.

```
/home/user/scripts/script_name.sh
```

4. Run with Sudo (if needed):

If the script requires administrator privileges, you can run it using sudo:

```
sudo ./script_name.sh
```

SYSTEM AND USER-DEFINED VARIABLES

In shell programming, **variables** are used to store data such as numbers, text, paths, and system information. Variables make programs flexible, reusable, and easy to manage.

Variables are mainly classified into:

1. System Variables
2. User-Defined Variables

1. System Variables

System variables are predefined variables created and maintained by the shell.

They store information related to the system environment, user details, and shell behavior.

These variables are automatically available when the shell starts.

Characteristics of System Variables

- Created by the operating system or shell
- Written in uppercase letters

OPERATING SYSTEM

- Accessible in all shell scripts
- Cannot be easily modified by users
- Used to get system-related information

HOME - Home directory of the current user

USER - Username of the logged-in user

PWD - Current working directory

SHELL - Path of the current shell

PATH - Directories where shell searches commands

LOGNAME - Login name

System variables are accessed using the \$ symbol.

echo \$HOME

echo \$USER

echo \$SHELL

2. User-Defined Variables

User-defined variables are variables created by the user to store values temporarily during program execution.

These variables are mainly used in shell scripts to store: Numbers, Strings, File names, User input

Rules for Creating User-Defined Variables

- No space on either side of =
- Variable names are case-sensitive
- Should not start with numbers
- No special characters except _

Accessed using \$variable_name

Syntax:

variable_name=value

Example:

name=Selvi

```
dept=Computer Applications
```

```
echo $name
```

```
echo $dept
```

Difference Between System Variables and User-Defined Variables

System Variables

- System variables are created by the shell itself.
- They store system-related information.
- They are mostly written in **uppercase letters**.
- They are available by default when the shell starts.
- Examples: HOME, PATH, SHELL

User-Defined Variables

- User-defined variables are created by the user.
- They store data given by the user.
- They are usually written in **lowercase letters**.
- They must be defined before use.
- Examples: name, age, salary

‘expr’ COMMAND

`expr` stands for “**expression**” and allows for the evaluation of values and returns the result to standard output. It is particularly useful in shell scripts for handling both numerical and string data efficiently.

Syntax: `$expr expression`

It supports

- Arithmetic operations
- String operations,
- Comparisons
- Logical comparisons.

Arithmetic Operations:

`expr` can be used to perform basic arithmetic calculations such as addition, subtraction, multiplication, division, and modulus.

- Addition:

OPERATING SYSTEM

```
result=$(expr 5 + 3)           # Adds 5 and 3
```

```
echo $result                   # Outputs: 8
```

- Subtraction:

```
result=$(expr 10 - 4)          # Subtracts 4 from 10
```

```
echo $result                   # Outputs: 6
```

- Multiplication:

```
result=$(expr 5 \* 2)          # Multiplies 5 and 2 (Note the escape character for *)
```

```
echo $result                   # Outputs: 10
```

- Division:

```
result=$(expr 10 / 2)          # Divides 10 by 2
```

```
echo $result                   # Outputs: 5
```

- Modulus (remainder):

```
result=$(expr 10 % 3)          # Gives the remainder when 10 is divided by 3
```

```
echo $result                   # Outputs: 1
```

String Operations:

It can find the length of a string, extract a substring, or match patterns.

- Length of a String:

```
result=$(expr length "Hello")  # Finds the length of the string "Hello"
```

```
echo $result                   # Outputs: 5
```

- Extracting a Substring:

```
result=$(expr substr "Hello, World" 1 5) # Extracts a substring starting at position 1 with length 5
```

```
echo $result                   # Outputs: Hello
```

Comparisons:

expr can be used for comparisons (like checking if two values are equal, greater than, or less than).

- Equal:

```
result=$(expr 5 = 5)           # Checks if 5 is equal to 5
```

```
echo $result                   # Outputs: 1 (True)
```

OPERATING SYSTEM

- Not Equal:

```
result=$(expr 5 != 3)          # Checks if 5 is not equal to 3
echo $result                    # Outputs: 1 (True)
```

- Greater Than:

```
result=$(expr 10 \> 3)         # Checks if 10 is greater than 3
echo $result                    # Outputs: 1 (True)
```

- Less Than:

```
result=$(expr 3 \< 10)         # Checks if 3 is less than 10
echo $result                    # Outputs: 1 (True)
```

Logical Operations:

It supports logical operations such as AND (\&) and OR (\|).

- AND Operation:

```
result=$(expr 1 \& 1)          # Logical AND between 1 and 1
echo $result                    # Outputs: 1
```

- OR Operation:

```
result=$(expr 0 \| 1)          # Logical OR between 0 and 1
echo $result                    # Outputs: 1
```

SHELL SCREEN INTERFACE

A shell screen interface is a text-based interface that allows users to interact with the operating system through a command-line shell. It provides a platform for executing commands, running programs, and automating tasks using scripts.

Key Components:

1. Command Prompt:

This is the symbol or string displayed by the shell to indicate it's ready to receive input. It often includes the user's name, the machine's hostname, and the current directory.

```
user@hostname:~$
```

2. Input Area:

Where users type their commands.

```
ls -l
```

3. Output Area:

After entering a command, the shell displays the output or any error messages.

Example (for ls -l):

```
-rw-r--r-- 1 user group 1234 Dec 26 10:00 file.txt
```

4. Cursor:

A blinking indicator that shows where the next character will appear in the input area.

5. Command History:

The shell keeps a record of previously entered commands. Users can access past commands using the up and down arrows.

‘read’ AND ‘echo’ COMMANDS

read COMMAND

The **read command** is used to **take input from the user** during the execution of a shell script. It reads data from standard input (keyboard) and stores it in a variable.

Syntax:

```
read variable_name
```

Example:

```
read name  
echo $name
```

Explanation:

The **read command** takes input from the user and stores it in the variable **name**. The **echo command** displays the value stored in the variable.

echo COMMAND

The **echo command** is used to **display text, variables, or output** on the screen. It sends the output to standard output (terminal).

Syntax:

```
echo message
```

Example:

```
echo "Hello World"
```

Output:

Hello World

Difference Between read and echo

read Command

- Used to take input from the user
- Reads data from keyboard
- Stores input in variables

echo Command

- Used to display output
- Prints data on the screen
- Displays text or variable values

COMMAND SUBSTITUTION

Command substitution it means running a command inside another command and using the result of the first command as if it were part of the second.

How Command Substitution Works:

When you execute a command in the shell, it produces output, which is typically printed to the screen. With command substitution, you can capture that output and use it in a different command.

There are two ways to perform command substitution:

1. Using Backticks (`):

```
result=`command`
```

In this syntax, the command inside the backticks is executed first, and its output replaces the backtick expression.

2. Using Dollar Sign and Parentheses (\$()):

```
result=$(command)
```

This method is preferred because it is easier to read, especially when commands are nested. It performs the same operation as the backtick syntax but is more flexible and visually clearer.

Why \$() is Preferred:

While both methods work, \$() is more readable, particularly in complex commands. It also avoids confusion when nesting commands.

For example:

OPERATING SYSTEM

- Nested Backticks (confusing):

```
result=`command1 \`command2``
```

- Nested \$() (clearer):

```
result=$(command1 $(command2))
```

Examples of Command Substitution:

```
1) current_date=$(date)
```

```
echo "Today's date is: $current_date"
```

```
2) echo "The current directory is: $(pwd)"
```

ESCAPE SEQUENCE CHARACTERS

Escape sequence characters are special characters used in programming and shell scripting to represent certain actions or symbols that cannot be easily typed or displayed directly in a string. Its representing non printable characters.

In escape sequences, the backslash (\) is the escape character that signals the start of a special character or sequence.

Common Escape Sequences:

1. \n - Newline

Moves the cursor to the next line.

Example:

```
echo "Hello\nWorld"
```

Output:

Hello

World

2. \t - Horizontal Tab

Inserts a tab space, which is often used for indentation.

Example:

```
echo "Name\tAge"
```

```
echo "Alice\t30"
```

Output:

OPERATING SYSTEM

Name Age

Alice 30

3. \ - Backslash

Represents a literal backslash character (\).

Example:

```
echo "This is a backslash: \\"
```

Output:

This is a backslash: \

4. ' - Single Quote

Inserts a single quote (') inside a string enclosed by single quotes.

Example:

```
echo 'It\'s a sunny day'
```

Output:

It's a sunny day

5. " - Double Quote

Inserts a double quote (") inside a string enclosed by double quotes.

Example:

```
echo "She said, \"Hello!\""
```

Output:

She said, "Hello!"

6. \r - Carriage Return

Moves the cursor to the beginning of the current line.

Example:

```
echo "Hello\rWorld"
```

Output:

World

(The "Hello" is overwritten by "World" because of the carriage return.)

7. \b - Backspace

Moves the cursor one position back, effectively deleting the previous character.

Example:

```
echo "Hello\bWorld"
```

Output:

HellWorld

Why Use Escape Sequences?

Escape sequences are crucial for:

- They help with organizing text, such as adding indentation (\t) or line breaks (\n).
- Representing characters that can't be typed directly, such as backslashes (\\) or quotes (\').
- Allowing color changes, bold text, and other effects in terminal output (\e).

SHELL SCRIPT ARGUMENTS

Shell Script Arguments are the values or data that are passed to a shell script when it is executed.

These arguments allow the script to accept input at runtime, making it more flexible and reusable.

[When you run a shell script from the command line, you can pass values (arguments) that the script can use. For example, if you pass a name and age to a script, the script can greet the person by name and tell their age.]

Positional Parameters are the variables used within the script to access these arguments, and they are automatically assigned based on the order in which the arguments are provided when running the script.

Common Positional Parameters:

1. \$0 - Script Name:

Refers to the name of the script.

```
echo "The script name is $0"
```

2. \$1, \$2, \$3, ... - Arguments:

These represent the arguments passed to the script, starting from \$1 for the first argument.

```
echo "First argument: $1"
```

```
echo "Second argument: $2"
```

3. \$# - Number of Arguments:

Represents the total number of arguments passed to the script.

```
echo "Total number of arguments: $#"
```

4. \$@ - All Arguments as Separate Items:

Represents all arguments passed to the script, preserving spaces and treating each argument as a separate entity.

```
echo "All arguments as separate items: $@"
```

5. \$* - All Arguments as a Single String:

Combines all arguments into a single string. Spaces between arguments are not preserved when using \$*.

```
echo "All arguments as a single string: $*"
```

TEST COMMAND

The test command in shell scripting is used to evaluate conditions, it returns a true (0) or false (1) result based on the condition. It is commonly used in if statements to control script flow based on the evaluated condition.

The test command is also commonly written using square brackets ([...]).

For example, test expression is equivalent to [expression].

Syntax:

```
test expression  
or  
[ expression ]
```

Common Test Expressions:

1. File Tests:

- -e file: True if the file exists.
- -f file: True if the file is a regular file (not a directory).
- -d file: True if the file is a directory.
- -r file: True if the file is readable.
- -w file: True if the file is writable.
- -x file: True if the file is executable.

2. String Tests:

- -z string: True if the string is empty.
- -n string: True if the string is not empty.
- string1 = string2: True if the two strings are equal.
- string1 != string2: True if the two strings are not equal.

3. Numeric Tests:

- num1 -eq num2: True if the numbers are equal.
- num1 -ne num2: True if the numbers are not equal.
- num1 -lt num2: True if num1 is less than num2.
- num1 -le num2: True if num1 is less than or equal to num2.
- num1 -gt num2: True if num1 is greater than num2.
- num1 -ge num2: True if num1 is greater than or equal to num2.

Unit 4

Chapter 3 - Control Structures

CONDITIONAL CONTROL STRUCTURES

These structures allow scripts to make decisions based on conditions.

1. if Statement

Executes a block of code if a condition is true.

Syntax:

```
if [ condition ];then
    # commands to execute if condition is true
fi
```

Example:

```
if [ $num -gt 0 ];then
    echo "Positive number"
fi
```

2. if-else Statement

Executes one block of code if the condition is true and another if it's false.

Syntax:

```
if [ condition ];then
    # commands if condition is true
else
    # commands if condition is false
fi
```

Example:

```
if [ $age -ge 18 ];then
    echo "You are an adult."
else
    echo "You are a minor."
fi
```

3. if-elif-else Statement

Used to check multiple conditions sequentially.

Syntax:

```
if [ condition1 ];then
    # commands if condition1 is true
elif [ condition2 ];then
    # commands if condition2 is true
else
    # commands if none of the conditions are true
fi
```

Example:

```
if [ $marks -ge 85 ];then
    echo "Grade: A"
elif [ $marks -ge 60 ];then
    echo "Grade: B"
else
    echo "Grade: C"
fi
```

4. case Statement

Matches a variable against multiple patterns.

Syntax:

```
case $variable in
    pattern1)
        # commands for pattern1;;
    pattern2)
        # commands for pattern2 ;;
    *)
        # commands if no patterns match;;
esac
```

Example:

```
echo "Enter a number between 1 and 3: "
read num
```



```
case $num in
    1) echo "You selected One." ;;
    2) echo "You selected Two." ;;
    3) echo "You selected Three." ;;
    *) echo "Invalid input." ;;
esac
```

LOOPING CONTROL STRUCTURES

These structures repeat a block of code as long as a condition is true.

1. while Loop

Executes a block of code repeatedly while a condition is true.

Syntax:

```
while [ condition ];do
    # commands to execute
done
```

Example:

```
count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done
```

2. until Loop

Executes a block of code repeatedly until a condition becomes true.

Syntax:

```
until [ condition ]
do
    # commands to execute
done
```

Example:

OPERATING SYSTEM

```
count=1
until [ $count -gt 5 ]
do
    echo "Count: $count"
    count=$((count + 1))
done
```

3. for Loop

Iterates over a sequence of values.

Syntax:

```
for variable in list
do
    # commands to execute
done
```

Example:

```
for num in 1 2 3 4 5
do
    echo "Number: $num"
done
```

JUMPING CONTROL STRUCTURES

These allow modifying the normal flow of loops.

1. break Statement

Exits a loop immediately.

Example:

```
for num in 1 2 3 4 5;do
    if [ $num -eq 3 ];then
        break
    fi
    echo "Number: $num"
done
```

Output:

Number: 1

Number: 2

2. continue Statement

Skips the current iteration of a loop and moves to the next iteration.

Example:

```
for num in 1 2 3 4 5;do
    if [ $num -eq 3 ]; then
        continue
    fi
    echo "Number: $num"
done
```

Output:

Number: 1

Number: 2

Number: 4

Number: 5

3. exit Statement

Exits the script entirely.

Example:

```
Echo "Enter a number (0 to quit):"
read num
if [ $num -eq 0 ];then
    echo "Exiting the script."
    exit
fi
```

Summary Table

Control Structure	Purpose	Example
if	Executes if a condition is true	Check if a number is positive
case	Matches variable with patterns	Execute based on user input
while	Repeats while condition is true	Print numbers from 1 to 5
until	Repeats until condition is true	Print numbers until greater than 5

for	Iterates over a sequence of values	Print items in a list
break	Exits a loop	Stop loop when a condition is met
continue	Skips the current iteration of a loop	Skip printing a specific number
exit	Terminates the script	Quit script when user enters 0

SHELL PROGRAMS COVERING THE ABOVE CONCEPTS

```
#!/bin/bash
```

```
# Menu-driven program demonstrating all shell control structures
```

```
while true; do
```

```
    echo "=====
```

```
    echo "Menu:"
```

```
    echo "1. IF Statement (simple)"
```

```
    echo "2. IF-ELSE Statement"
```

```
    echo "3. IF-ELIF-ELSE Statement"
```

```
    echo "4. CASE Statement"
```

```
    echo "5. FOR Loop"
```

```
    echo "6. WHILE Loop"
```

```
    echo "7. UNTIL Loop"
```

```
    echo "8. BREAK & CONTINUE"
```

```
    echo "9. EXIT Script"
```

```
    echo "=====
```

```
    echo "Enter your choice (1-9): "
```

```
    read choice
```

```
    case $choice in
```

```
        1)
```

```
            # Simple IF
```

```
            read -p "Enter a number: " num
```

```
            if [ $num -gt 0 ]; then
```

```
    echo "$num is positive"
fi
;;
```

2)

```
# IF-ELSE
echo "Enter a number: "
read num
if [ $((num % 2)) -eq 0 ]; then
    echo "$num is even"
else
    echo "$num is odd"
fi
;;
```

3)

```
# IF-ELIF-ELSE
echo "Enter a number: "
read num
if [ $num -gt 0 ]; then
    echo "$num is positive"
elif [ $num -lt 0 ]; then
    echo "$num is negative"
else
    echo "Number is zero"
fi
;;
```

4)

```
# CASE
echo "Enter a letter (a/b/c): "
read letter
case $letter in
    a) echo "You entered a";;
    b) echo "You entered b";;
    c) echo "You entered c";;
    *) echo "Invalid input";;
esac
```

```
;;
```

5)

```
# FOR loop
echo "Enter N for FOR loop: "
read N
for i in $(seq 1 $N); do
    echo "FOR Loop Number: $i"
done
```

```
;;
```

6)

```
# WHILE loop
i=1
echo "Enter N for WHILE loop: "
read N
while [ $i -le $N ]; do
    echo "WHILE Loop Number: $i"
    i=$((i+1))
done
```

```
;;
```

7)

```
# UNTIL loop
i=1
echo "Enter N for UNTIL loop: "
read N
until [ $i -gt $N ]; do
    echo "UNTIL Loop Number: $i"
    i=$((i+1))
done
```

```
;;
```

8)

```
# BREAK & CONTINUE example
for i in 1 2 3 4 5 6 7 8 9 10; do
    if [ $i -eq 3 ]; then
        echo "Skipping $i (continue)"
        continue
    fi
done
```

OPERATING SYSTEM

```
        fi
        echo "Loop Number: $i"
        if [ $i -eq 7 ]; then
            echo "Breaking loop at $i"
            break
        fi
    done
;;
9)
# EXIT
echo "Exiting script..."
exit 0
;;
*)
echo "Invalid choice. Try again."
;;
esac
echo
done
```

OUTPUT

=====

Menu:

1. IF Statement (simple)
2. IF-ELSE Statement
3. IF-ELIF-ELSE Statement
4. CASE Statement
5. FOR Loop
6. WHILE Loop
7. UNTIL Loop
8. BREAK & CONTINUE
9. EXIT Script

=====

Enter your choice (1-9): 1

Enter a number: 5

OPERATING SYSTEM

5 is positive

=====

Enter your choice (1-9): 2

Enter a number: 4

4 is even

=====

Enter your choice (1-9): 3

Enter a number: -2

-2 is negative

=====

Enter your choice (1-9): 4

Enter a letter (a/b/c): b

You entered b

=====

Enter your choice (1-9): 5

Enter N for FOR loop: 3

FOR Loop Number: 1

FOR Loop Number: 2

FOR Loop Number: 3

=====

Enter your choice (1-9): 6

Enter N for WHILE loop: 3

WHILE Loop Number: 1

WHILE Loop Number: 2

WHILE Loop Number: 3

=====

Enter your choice (1-9): 7

Enter N for UNTIL loop: 3

UNTIL Loop Number: 1

OPERATING SYSTEM

UNTIL Loop Number: 2

UNTIL Loop Number: 3

=====

Enter your choice (1-9): 8

Skipping 3 (continue)

Loop Number: 1

Loop Number: 2

Loop Number: 4

Loop Number: 5

Loop Number: 6

Loop Number: 7

Breaking loop at 7

=====

Enter your choice (1-9): 9

Exiting script...